CSE360: Computer Interfacing

Lab Project Report

Title: Smart Earthquake Survival Detection

Group_No: 02
Section:10

Group Members:

Name	ID
Sikder Marbin Rime	22301288
Labiba Zaman	22201881
Kaiser Fahim	22301424
Lamia Nasrin Prity	22299052

Department of Computer Science and Engineering
BRAC University

DATE:18/08/26

Abstract

Earthquakes are the most unpredictable natural disaster as there is no way to predict them. Following an earthquake, it is a critical situation for rescue operations for dense countries such as Bangladesh where it is important to know where to search first. We have developed the Smart Earthquake System which is a low cost prototype embedded system using Arduino Uno for real-time monitoring of earthquakes with an MPU6050 sensor and after confirming a strong earthquake, it scores three different zones for survivor activity by combining PIR, sound and temperature sensors. A second shake activates a Red alert LED, a beep sequence and opens a 1 minute scanning window in which each zone collects a survivor-activity score from its sensors. The scan is displayed on the 16x2 I2C LCD at the end of the scan as the "rescue priority", plus the raw temperature/PIR/sound counts.we can reset the entire system with a push button. This is a prototype that has been constructed and tested on a breadboard. Issues encountered were I2C bus reliability, sensor polarity and threshold tuning, which were all identified using an I2C scanner and corrected by re-wiring and software changes. A wireless reporting system might also be incorporated in the future, but this is a multi-sensor data-fusion project with a low cost for prioritizing the disaster-rescue scenario.

Keywords

Embedded systems, MPU6050, sensor fusion, survivor rate, I2C communication, earthquake detection.

1. Introduction

Earthquakes occur suddenly, and when it happens, rescue teams have limited resources and time to conduct a search of several affected areas. This is one of the most critical decisions that can affect the life of people trapped inside a collapsed or damaged building, and can often be made without any sensor-based information on the scene of first response.

Earthquake detection: using the accelerometer sensor to measure the amount of shaking, the alert is escalated in steps (Normal to Warning to Alert) as the shaking is confirmed, the three zones monitored for one minute only have a fixed monitoring window (motion, sound and heat), a simple and transparent survivor likelihood score is calculated for each zone based on the sensor readings, and the zone with the highest survivor likelihood is immediately displayed on a local LCD and buzzer alert.

The project is carefully designed so as to focus on the real heart of the Computer Interfacing course: interfacing multiple sensors (digital and analog), multiple peripherals (one MPU, one LCD), a limited number of GPIOs, and an effective real-time control loop that integrates sensing, decision logic and multi-modal, non-blocking output (LEDs, buzzer, LCD).

2. Related Work/ Inspiration

Commercial earthquake early-warning systems that detect P-waves travelling through the ground, and alert people before the subsequent S-waves, are based on distributed seismometers, like the regional seismic networks in Japan. The concept of these systems has been scaled down here to a single low-cost (small budget) IMU (MPU6050) to be used as an earthquake trigger based on vibration thresholds measured by an accelerometer.

Common projects are consumer PIR-based motion / alarm systems, and simple sound activated alarm modules . This project uses the same approach to sensor interfacing but the same types of sensors (PIR, sound, and temperature) for evidence of survivor presence, not for intrusion detection.

Differentiation: Many earthquake detector projects that are done end with the vibrating motion being detected and an alarm being sounded. This project takes one more step: An inexpensive, widely-available module is used to estimate the likelihood of survival by zone motion, sound, and heat-hazard signals combined into a 0-100 likelihood of survival rating is added to the LCD, making a generic alarm an actionable triage aid.

3. Technical Approach

System Architecture:

- The Arduino Uno board is the control center of the whole system, each sensor and output connected to it.
- It speaks with an MPU6050 sensor over I2C (using the Adafruit_MPU6050 library) and continually polls, continually, the amount of shaking!
- The alarm is continuously increasing in severity from Normal (GREEN) to Warning (YELLOW, 1 low beep) and then to Alert (RED, 5 beeps and begins a 1 minute fixed survivor scan window).
- For one minute, the Arduino monitors three distinct "zones" (as if three rooms) each equipped with three small sensors: a PIR sensor (motion), a sound sensor (noise/alarm of help), and a temperature sensor (heat/fire alarm).
- Readings in each zone are collected so that the score is a running total of the amount of survivor activity in that zone throughout the minute - not just for one moment - PIR and Sound each add +1 if triggered, Temperature subtracts 1 when it reads “hot”.
- At the end of the minute, a 2-beep sequence is heard and the zone with the greatest score appears on the 16x2 LCD as “Zone X Priority” with all 3 zones Temp/PIR/Sound breakdown, also all 3 zones Temp/PIR/Sound breakdown will be printed to the Serial Monitor.

- A reset button is provided to enable the entire system to function from the beginning.

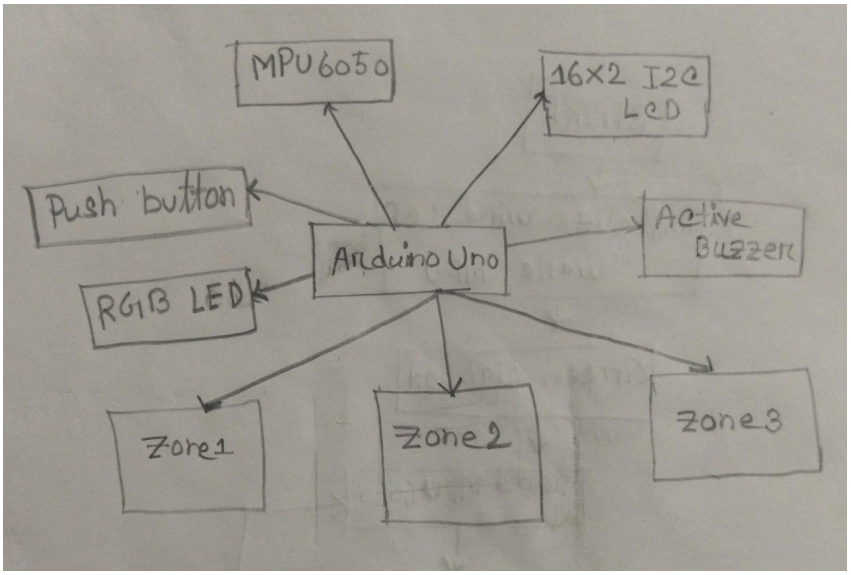
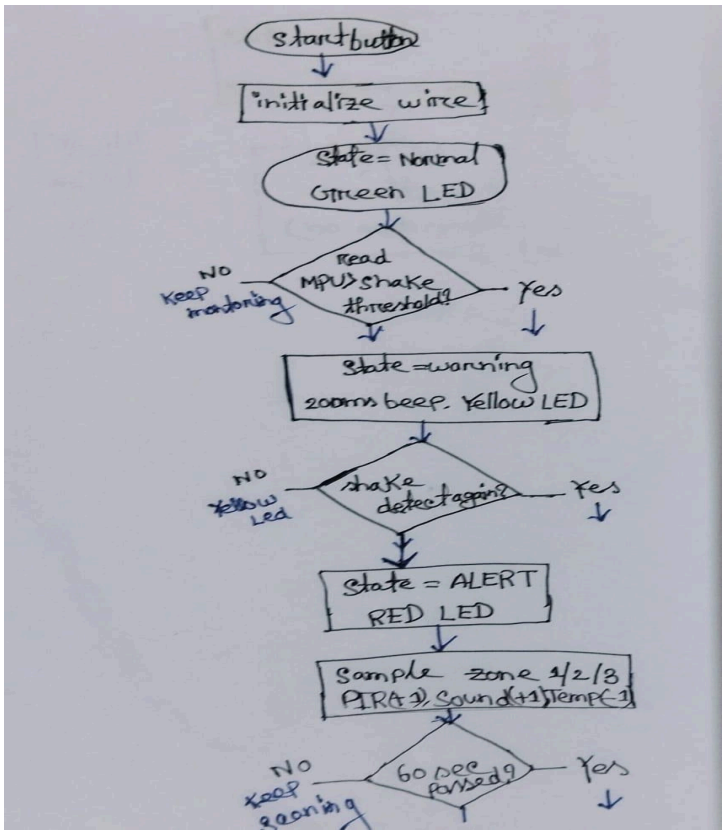


Fig1: System Block Diagram



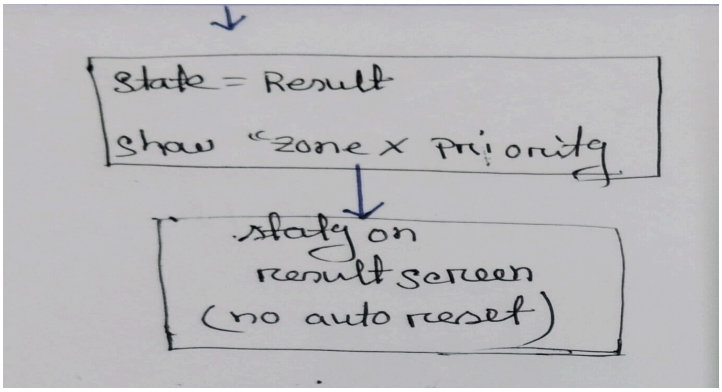


Fig2: System Flowchart

Components Used:

Component name	work
Arduino Uno	Main part of the system. It is connected to the all other parts and act like head
MPU6050	It measures the shaking,to detect earthquakes we use it.
PIR Sensor	It detect the motion of each zones it person is moving
Sound Sensor	To detect the sound of each zone like as someone is calling for help
Temperature Sensor	It detect the hot like possibility of fire risk
LEDs(red,green,yellow)	Green light shows everything is normal ,yellow light shows when the state is in warning position,red light shows when an earthquake has happened.
Active buzzer	Sound when an earthquake is detected.
Push Button	Reset the whole system.
Breadboard	Set all the led ,registers, power everything
Jumper wires	Used to connect the components
Battery	Power source…

Cost Breakdown:

Item	Qty	Price
Arduino Uno	**1**	**550**
MPU6050	**1**	**372**
PIR Motion Sensor	**1* 3**	**276**
Sound Sensor	**1*3**	**219**
Temperature Sensor	**1*3**	**285**
LEDs(red,green,yellow)	**3**	**15**
Active buzzer	**1**	**52**
Push Button	**1**	**5**
Breadboard	**2**	**210**
Jumper wires	~	**100**
Battery	**2**	**74**
Total(approx)		**2158 Tk**

Communication Protocol:

- I2C :used to connect both the MPU6050 sensor and the LCD screen. I2C requires just 2 wires (SDA and SCL) and both devices use the same 2 wires (A4 and A5 of the Arduino).We chose I2C as we needed a lot of pins for the 3 zones of sensors, the LEDs and the buzzer and thus I2C saved us some pins since we could have 2 devices using 2 wires. The MPU6050 is accessed via the Adafruit_MPU6050 library, which still uses the same I2C bus.

• Digital I/O (GPIO): For PIR sensors, sound sensors, temperature sensors and LEDs/buzzer. These parts can be read or controlled with a simple ON/OFF (HIGH/LOW) signal, which is the easiest and fastest method to read/control these parts (no special protocol required).

During the development of the project we used Serial (USB) to communicate with the computer at 9600 bauds, with the Arduino printing out the current sensor values and final zone scores. This facilitated us to identify and resolve a number of issues in testing.

Circuit/Schematic:

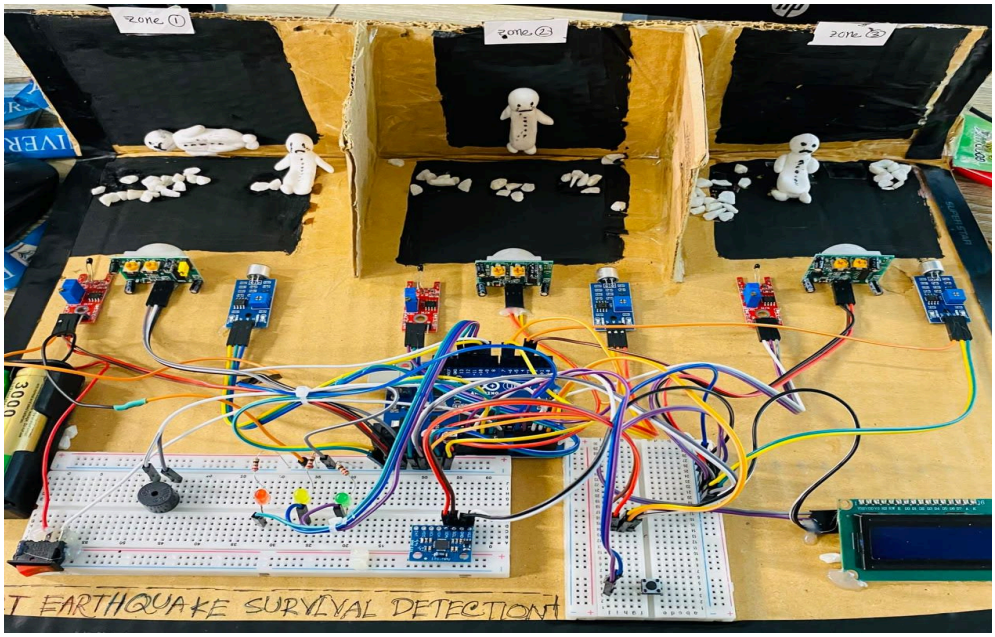


Fig3: System diagram

Implementation:

- On startup, the Arduino initializes the LEDs/buzzer pins, initializes the LCD, and invokes the mpu.begin(); function which will display “MPU6050 Error!” on the LCD if the MPU6050 is not present, and the program will halt.
- The range of the accelerometer, gyro and filter bandwidth are set once and the system will enter the NORMAL state (Green LED on, LCD display: Status: Normal / Monitoring...).

- In NORMAL, the Arduino reads the MPU6050 data each loop, if the data is more than SHAKE_THRESHOLD away from 9.8 m/s², the system jumps to WARNING (Yellow LED, one 200ms beep) and stays in WARNING until the deviation from 9.8 m/s² is less.The Arduino reads the data from the MPU6050 every loop and if the data is more than SHAKE_THRESHOLD away from 9.8 m/s² the system jumps to WARNING (Yellow LED, one 200ms beep) and remains in WARNING until the deviation from 9.8 m/s² is less.
- In WARNING, a second confirmed shake will switch the system to ALERT (Red LED), reset all zone scores to 0, and trigger a 5-beep alert sequence and a 60 second scan window.
- In ALERT, each 200ms the code polls all three zones; PIR HIGH gets +1, Sound HIGH gets +1, and the analog temp that is below the threshold subtracts 1 (65). Each zone's running score is the survivor-activity score.
- Refreshes LCD every 500ms while scanning, displays live countdown and active score for each zone (Z1/Z2/Z3).
- When the 60 seconds are finished, the system goes into RESULT, a 2-beep sequence is played once, the Zone with the highest total score is displayed and the LCD displays “Zone X Priority” with the Temp/PIR/Sound breakdown, and the full breakdown for all Zones is printed on the Serial Monitor.
- All the beep patterns (single Warning beep, 5x Alert beeps, 2x Result beeps) are all implemented without delay(), with timers based on millis(), and so the LCD, sensors, and state logic will run smoothly while the beep pattern is playing in the background.

Challenges:

- Initially, MPU6050 sensor was not giving any data. Created a small "I2C Scanner" program to test it, and discovered a short circuit on the breadboard and soldered it with a hard wire.
- The alarm was not sensitive enough, the touch of one shake triggered the alarm. We solved this by introducing two new stages - Warning at 1st shake, Alert at 2nd shake and adjusting the shake amount by observing the values of the sensors as they happen on Serial Monitor.
- Delay() was used for buzzer beeps, but this caused the entire program to freeze and sensors and LCD not to update during a buzzer beep. We replaced the beeps with millis() types, so that they play in the background without blocking anything.
- One reading of each zone was too random. This was resolved by scanning for 60 seconds and by checking every 200ms, hence capturing real activity rather than a lucky/unlucky moment.

- The temperature sensor had read the opposite of what we wanted, so we modified the code to call for the temperature to go below the threshold.
- Since there are only few pins available on the Arduino Uno, the pin mapping of 3 zones, 3 LEDs and the buzzer was carefully planned. We also changed the library that we used to get the sensors readings from so that it was the Adafruit_MPU6050 library, which means sensor readings are more reliable.

4. Sustainability & Impact

Sustainability

It consumes very little power (few hundred milliamps running on 5V) which means that it can operate for much of the day on a small power bank or solar charged battery. This can be useful in areas where there is no regular power, such as immediately after an earthquake. All parts are reusable, not disposable, and normal, so they can be used in other projects in the future, reducing electronic waste.

Impact

This system converts raw data from the sensors into a single question: In which zone do survivors most likely exist? This can assist a small rescue team or a school safety drill or first responders in low-resource, earthquake-prone regions (such as in Bangladesh) to search faster than traditional alarm. It also has learning value as it demonstrates to the students of Computer Interfacing how to interface several sensors with one another, share an I2C bus, and how to create an embedded realtime system.

Future Work

- Get a more accurate temperature reading (DS18B20, LM35, etc. rather than HOT/OK).
- Include wireless reporting (such as ESP32 Wi-Fi) to enable zone data to be viewed on a phone or computer.
- Include SD card logging for data to analyze later.
- Use more than 3 zones with I2C expander or on a larger board (Arduino Mega / ESP32).

Limitations:

The current build does not have a wired reset button, after a scan is completed, the LCD will display the RESULT screen until the board is power cycled, the comparator and analog thresholds type of sound and temperature sensors are not a true measurement, but just a threshold detection, breadboard prototyping has caused repeated issues with connection reliability.

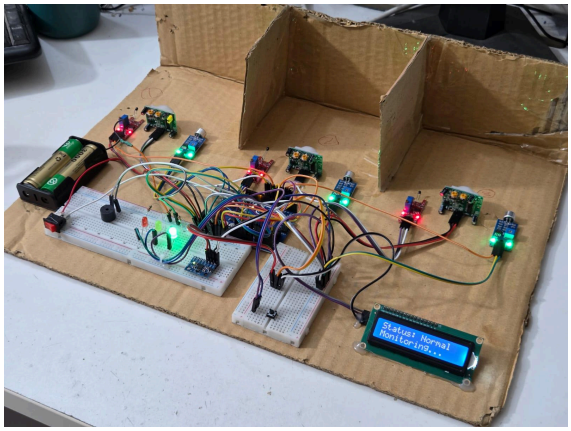
the +1 motion, +1 sound, −1 heat is an illustrative heuristic for purposes of demonstration only; scores are not a normalized 0–100% indicator of how accurately the motion was detected, but rather are only meaningful when compared against each other within the same scan, not as a quantitative measurement of how well the motion was detected; any single accelerometer can not distinguish a seismic epicenter location or differentiate a real earthquake from all other strong local vibrations (e.g., someone bumping the table).

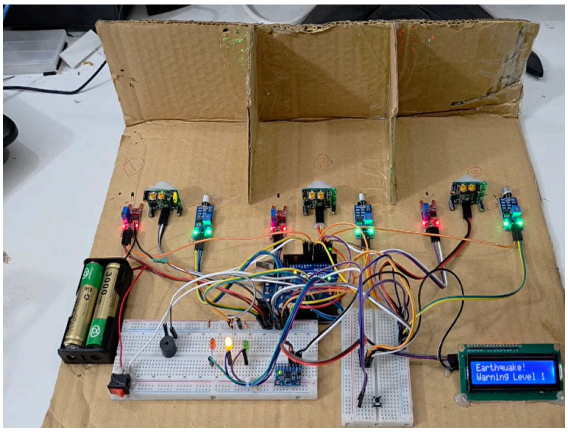
5. Results & Discussion

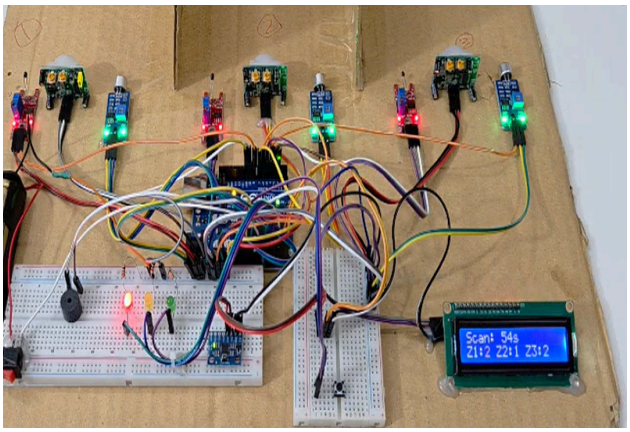
The system is a smart earthquake system, using Arduino Uno and MPU6050 and was tested successfully and all the modules were found to work properly. The logic to detect a shake worked correctly: Normal to Warning on first detected shake, then Warning to Alert on second detected shake, with no false triggers after tuning the threshold on light touches. All three zones' PIR, sound and temperature sensors were sampled every 200ms during the one minute Alert scan, and gave a live changing score on the LCD. The system correctly identified and displayed the zone with the highest survivor-activity score at the end of the scan and its Temp/PIR/Sound breakdown was also verified on the Serial Monitor, as was the same breakdown. Overall, the system achieved its objectives, offering a staged earthquake detection supported by a ranked earthquake rescue priority by zone.

Test case	input	Expected output	status
Normal monitoring	No shake	Green LED active, LCD shows “normal/monitoring”	PASS
First shake (warning)	One confirmed shake on MPU6050	Yellow LED on, 200 ms beep and shows “Earthquake!/Warning”	PASS
Second shake	Second confirmed shake while in warning	Red LED On, Lcd shows “major Alert!/scanning…”	PASS
Zone scanning	Active 3 sensor for all zones during scan		PASS
MPU6050	Sensor not found/disconnected	The LCD shows “MPU ERROR!”	PASS
Zone priority	60 sec screen	2x beep and LCD shows T,P,S breakdown with zone	PASS

		x priority	
Temperature sensor tigger	Heat near a zone's temp sensor	Temp score will be decrease (-1)	PASS
Reset system	Result screen has been shown	Require power to return a normal states	PASS







6. Conclusion

The aim of this project was to develop an inexpensive embedded platform which could detect earthquake vibration and return an actionable estimate of survivor activity by zone to aid rapid post-earthquake triage. The prototype is complete with an MPU6050 accelerometer, nine zone sensors across three individually scanned zones, and a buzzer/red, green, blue LED alarm, all communicating via a common I2C bus and direct GPIO, with a one-minute automated scan of survivor activity and a ranked "Zone X Priority" alarm recommendation displayed on a 16x2 I2C

LCD. A series of iterative tests and debugging, with a custom I2C scanner, two stage threshold calibration, and switching to non-blocking, millis()-based beep sequences, resulted in a system that could be relied upon to distinguish between normal and actual shaking and provide a clear suggestion for which zone survivors might be found. The project also serves as an example of the importance of low power, modular electronics for disaster response applications in resource-poor environments, and provides specific guidelines for how this classroom prototype can be improved to better support disaster response: a wired reset control, a wireless reporting system, calibrated sensing, and data logging.

7. **Contribution**

Name	ID	Role(s) / Responsibilities	Specific Contributions
Sikder Marbin Rime	22301288	Hardware Design, Circuit Building & Sensor Integration	Constructed and designed circuit with Arduino Uno on bread board. • Connected and tested the working of the MPU6050 sensor, PIR Sensor, Sound sensor, temperature sensor, LEDs, buzzer and push button. • Made 3 zone planning of monitoring zone Pin connections. • Helped troubleshoot breadboard connection and wiring problems. • Tinkered with I2C connection problem with MPU6050 with I2C scanner. • Performed hardware verification tests and reported their successful outcome.
Labiba zaman	22201881	Software Design, Programming & System Logic	• Wrote main program in Arduino for the earthquake detection system. - Added Normal -> Warning -> Alert -> Result state behaviour. Programmed Shake Detection/Threshold Handling using the MPU6050. • Added the logic to create survivor lists for 60 seconds and generate zones scoring. • Programmed LCD, LED and buzzer outputs. • Contributed to the fine tuning of the detection level for earthquakes and corrected the program in general.

Kaiser Fahim	22301424	Sensor Testing, Data Collection & Debugging	Tuned the PIR, sound and temperature sensors for each of the 3 zones. • Assisted in repeated testing for determining appropriate sensor thresholds. • Monitored and viewed sensor readings and zone scores using the Serial Monitor. Tested the 200ms sensor sampling process in a 60s scanning window. • Assisted in locating and resolving temperature sensor polarity/threshold problem. • Tried to create various earthquake/agitation conditions to minimize false alarms. Richard Carrow, Chief Curator of Natural History at the Bardarbunga Museum in Røros, Norway, was responsible for this.
Lamia Nasrin Prity	22299052	LCD/Communication, Testing, Documentation & Presentation	• Made contributions to the integration with the 16×2 I2C LCD and the display outputs. • Assisted with testing of the I2C communication between the Arduino, MPU6050 and LCD. • Validated the LCD display messages in Normal, Warning, Alert, scanning and Result modes. • Assisted in full system testing with the test cases established. Documented and compiled the project outcomes, perceptions and obstacles. • Supported in compiling the project report, the system explanation and presentation materials. • Assisted with the validation of the final demonstration and entire system.

References:

1. Bigyan Project. (2025, May 19). Science Earthquake Detector Project with Arduino. https://www.facebook.com/reel/1057766512911890

2. Christ, V. (2017). *Real-time sensor-based motion capture system for virtual reality* (Doctoral dissertation, Universität Koblenz).